

COMPX304-23A

Advanced Networking and Cybersecurity

Week 8
Application Security vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



Who are the attackers?

- Nation states
 - US, China, Russia are prominent in the news as they engage in offensive operations but almost every country now has a cyber arm of military that conducts offensive operations
- Cybercriminals
 - Almost half of the crime in developed countries is cybercrime (by volume and by value)
 - Money from state-run cybercrime is almost 8% of the GDP of North Korea.
 - Cybercriminals attack vulnerable systems and compromise them to
 - Demand ransom
 - Use the systems for other attacks
- The Swamp (as Ross Anderson puts it)
 - Attacks for personal and sexual abuse, hate campaigns, cyber bullying, etc.



User input

- Any application that takes user input is prone to attacks
- When we write applications we assume the user will input the data in the way and form the application expects!!
- Writing an application to add two numbers
 - We expect user will give the application two **numbers**



Add two numbers and use the sum for division

[illegible]

Example runs

[illegible][illegible]

Injection attacks

- Information that is processed by computers can roughly be divided into two types
 - Data
 - Information that is processed
 - Instructions
 - Tell a computers how to process that data



Add two numbers and use the sum for division

[illegible]

Injection Attacks

- In our programs we usually expect
 - the user to provide the **data** while
 - the program code itself is a **combination of instructions and data**
- An injection attack is an attack where the **attacker manages to input instructions along with the input data**
- This results in the program doing something not intended by the programmer
- Injection attacks affect programs written in interpreted languages



SQL Injection

8.3M records of Freepik and Flaticon users stolen in SQL injection attack



BY DUNCAN RILEY

Data relating to 8.3 million users of stock-image sites Freepik and Flaticon, both owned by Freepik Co. S.L., have been stolen through an SQL injection attack.

The data stolen included the email addresses of users along with 3.77 million hashed passwords. Some 3.55 million of those passwords were encrypted with bcrypt, making them highly difficult but not impossible to crack, while 229,000 were salted MD5, an older encryption standard that can be easily decrypted. All users with the latter have had their passwords reset and affected users were sent an email encouraging them to change their password if it was used on another site.

<https://siliconangle.com/2020/08/24/8-3m-records-freepik-flaticon-users-stolen-sql-injection-attack/>



Asim Rahal, Independent Cloud Security Consultant
September 2, 2020

Share



Surging CMS attacks keep SQL injections on the radar during the next normal

Every year, millions of websites across the world fall victim to malware attacks that are designed to gain access to the site's backend without the administrator's knowledge in order to steal sensitive data or cause damage, usually for financial gain. This year, cyberattacks **have been on the rise** during the pandemic, leaving businesses to wonder whether or not things will settle down whenever the COVID-19 situation begins to wane, or if this is the next normal for the indefinite future.

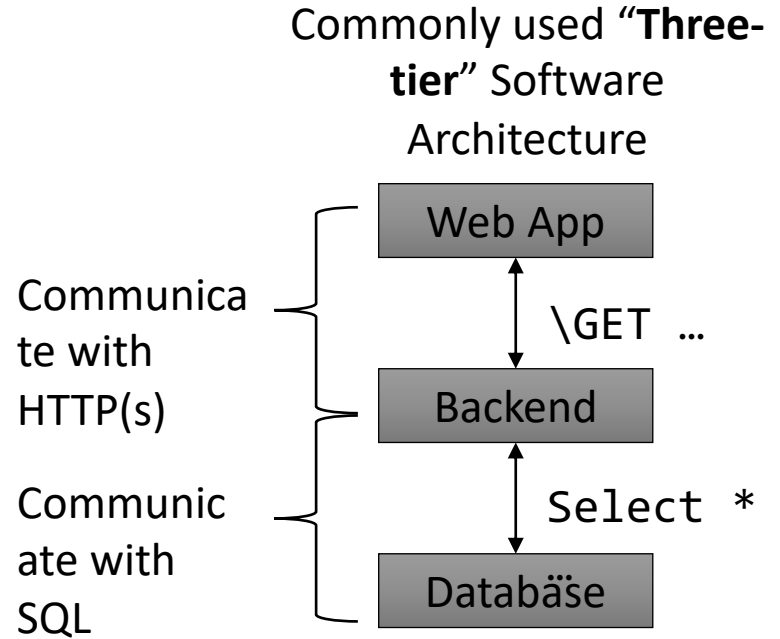
<https://www.helpnetsecurity.com/2020/09/02/surging-cms-attacks-keep-sql-injections-on-the-radar-during-the-next-normal/>



CROW Lab *Cybersecurity Researchers of Waikato*

SQL Injection

- Structured Query Language (SQL)
 - Used to send commands to a database
- SQL Injection
 - Malicious user injects their own SQL, which is executed as is by the DB



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```

Expected Usage:

- User types a valid id, for example "5"
- The query finds the name of user with id 5



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```

Way to attack the system:

- User types a more complicated string for id, for example:
""); DROP TABLE users;"
- This creates **two** queries one after the other
- The first query is badly formatted and it fails
- But, the second, separated by semi-colon, executes and ...



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```

"\"; DROP TABLE users;" is a rather naïve attack...

How about:

```
"\"; INSERT INTO users(username, password, isAdmin) VALUES  
(\"newRoot\", \"\", TRUE);"
```

And the attacker just “rooted” the whole website!!



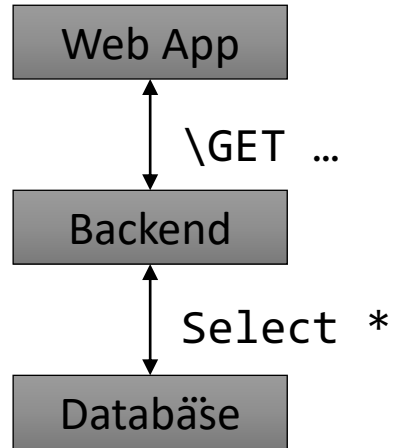
SQL Injection

- `SELECT count(name) FROM users WHERE name="{}`
 - Putting Vimal here works
 - So does `" or ""=""`; --

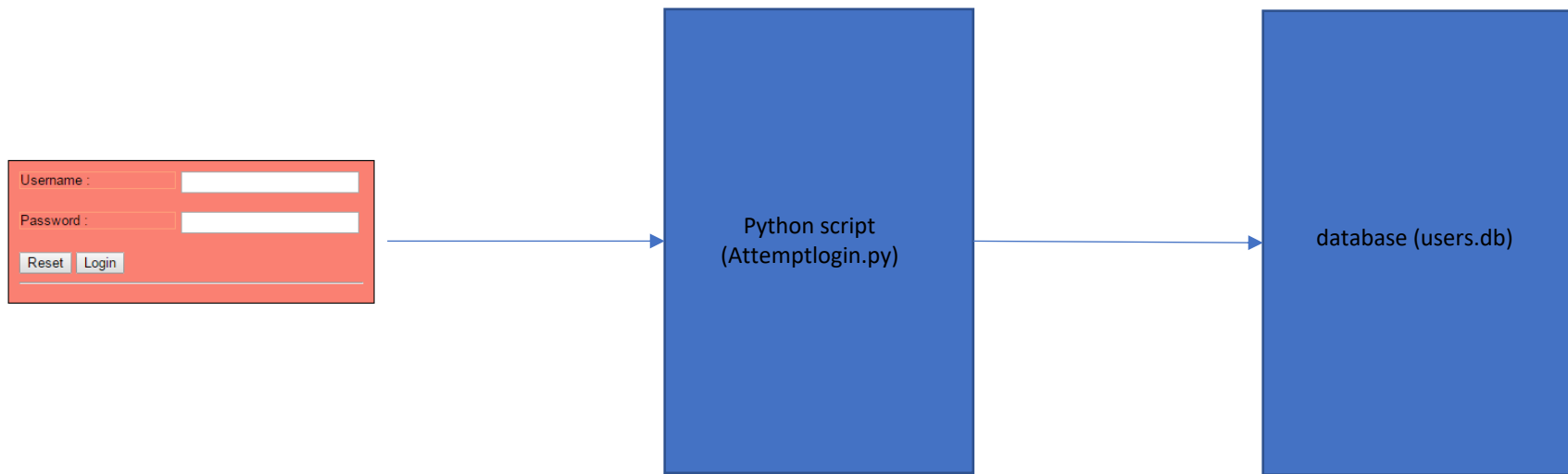


Three tier architecture

Commonly used “**Three-tier**” Software Architecture



Webpage with user input fields



`select * from users where username='{}' and password=x'{}'`

id	permissions	username	Password
0	1000	admin	ajkbhai2e3bjhbjh
1	500	bob	njbjkdnkj28829
2	10	alice	10nh7b8fff2ffffff3
3	5	root	128hbghiyvdi946



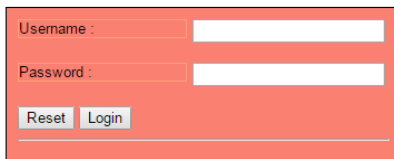
Login logic

- Correct username and password combination will select exactly one row
 - If no row is selected it means the username and password combination was incorrect
 - Appropriate action is taken in response to this such as letting the user know the username or password was wrong.
- If a row is selected as a result of the query
 - The username and password combination is correct and we can proceed with login
 - This usually transfers the user to a personalized page etc.
 - In the lab example, the user is showed the id and permission level.



Query examples

```
select * from users where username='{}' and password=x'{}'
```



Username :

Password :

```
select * from users where username='admin' and password=x'ajkbhai2e3bjhbjh'
```

```
select * from users where username='mario' and password=x'som password'
```



SQL injection

- The objective of sql injection is to inject sql query fragments as part of input data to
 - Make the query behave differently to what the programmer intended
 - In the case of a login page
 - The programmer intended the logic to work as previously described
 - Can we inject sql so that we can login without knowing the right username password combination

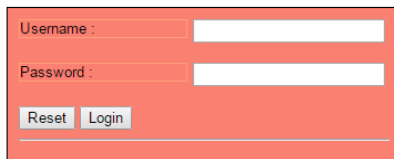


SQL injection

- One of the popular query fragments is

- ' OR '='; --

select * from users where username='{}' and password=x'{}'



Username :

Password :

select * from users where username= ' ' OR '='; -- ' and password=x'somegarbage'



```
select * from users where username= ' ' OR ' ' = ' ' ; -- ' and password=x'somegarbage'
```

```
select * from users where username= ' ' OR ' ' = ' ' ; -- ' and password=x'somegarbage'
```

```
select * from users where  
    username= ' ' OR ' ' = ' ' ;
```

```
select * from users ;
```



Parameterised queries

- Parameterised queries can mitigate sql injection to a large extent
 - Parameterised queries are compiled not interpreted
 - In parameterised queries the construction of a SQL statement containing user input is performed in two steps:
 1. Specify the query's structure, leaving placeholders for each item of user input.
 2. Specify the contents of each placeholder



Lab 7

```
select * from users where username = ? and password =x'{'
```

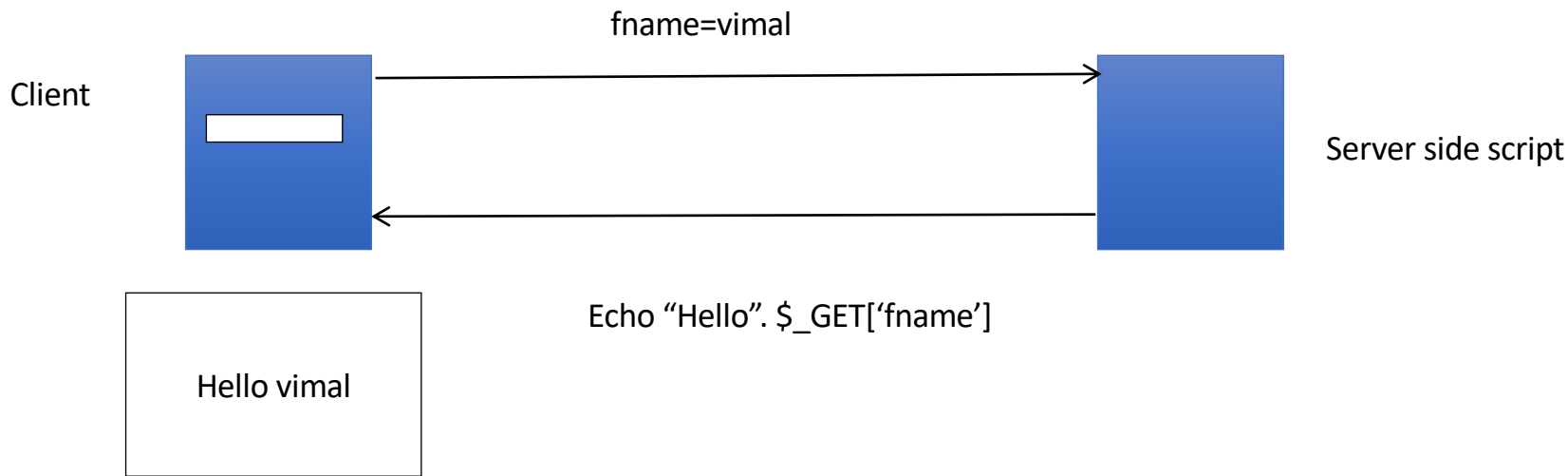
```
args = (user, )
```

user variable has previously been defined to be a string, so the input will be escaped to be a string



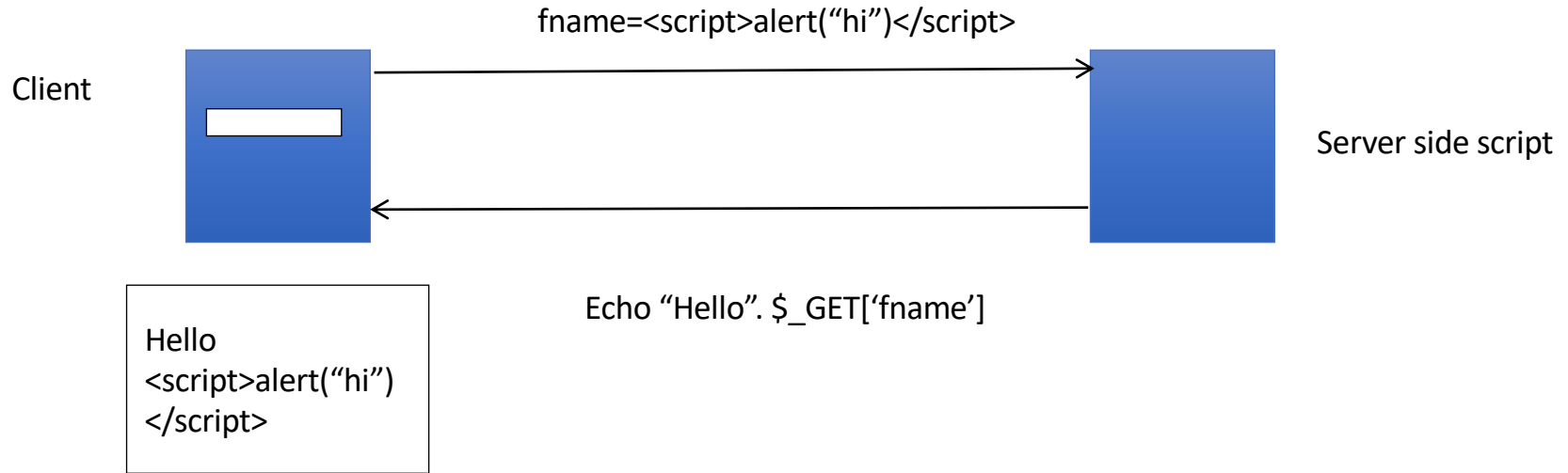
Cross Site Scripting

Cross-site scripting (XSS) is an injection attack that allows an attacker to execute malicious JavaScript in another user's browser



Cross Site Scripting

- If the attacker injects javascript code in the input field



How can an attacker embed JS in my URLs?

- Phishing (Social Engineering)
 - User receives a legitimate looking email with a URL which the email wants you to click on the pretext of
 - Account activation-deactivation/Shopping offers/Information from Bank/IRD/Police etc.
- URL shorteners are scary
 - <http://TinyURL/abcdef>
 - Who knows what is actually behind this URL



Malicious Javascript

- JavaScript has access to some of the user's sensitive information, such as cookies.
- JavaScript can send HTTP requests with arbitrary content to arbitrary destinations by using AJAX and other mechanisms.
- JavaScript can make arbitrary modifications to the HTML of the current page by using DOM manipulation methods.



Malicious Javascript

- Cookie theft
 - The attacker can access the victim's cookies associated with the website using `document.cookie`, send them to his own server, and use them to extract sensitive information like session IDs.
- Keylogging
 - The attacker can register a keyboard event listener using `addEventListener` and then send all of the user's keystrokes to his own server, potentially recording sensitive information such as passwords and credit card numbers.
- Phishing
 - The attacker can insert a fake login form into the page using DOM manipulation, set the form's action attribute to target his own server, and then trick the user into submitting sensitive information.



Session Hijacking

